

OS Lab

Skill-6

To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.

To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_ARGS 100
#define MAX_LEN 1024

int parse_command(char *input, char *args[])
{
    int argc = 0;
    int i = 0;
    int j = 0;

    char token[MAX_LEN];

    while (input[i] != '\0')
    {
        // Skip spaces
        while (isspace(input[i]))
        {
            i++;
        }

        if (input[i] == '\0')
        {
            break;
        }

        j = 0;

        while (input[i] != '\0' && !isspace(input[i]))
        {
```

```

    // Handle escape character
    if (input[i] == '\\')
    {
        i++;

        if (input[i] != '\\0')
        {
            token[j++] = input[i++];
        }
    }
    else
    {
        token[j++] = input[i++];
    }
}

token[j] = '\\0';

args[argc] = malloc(strlen(token) + 1);

if (args[argc] == NULL)
{
    perror("malloc");
    exit(1);
}

strcpy(args[argc], token);

argc++;

if (argc >= MAX_ARGS - 1)
{
    break;
}

args[argc] = NULL;

return argc;
}

int main()
{
    char input[MAX_LEN];

```

```
char *args[MAX_ARGS];

printf("Shell Parser\n");
printf("Enter command: ");

fgets(input, sizeof(input), stdin);

input[strcspn(input, "\n")] = '\0';

int argc = parse_command(input, args);

printf("\nParsed output:\n");

for (int i = 0; i < argc; i++)
{
    printf("Argument %d: [%s]\n", i + 1, args[i]);
    free(args[i]);
}

return 0;
}
```

```

lahari@DESKTOP-AIC33F7:~$ cd ~/OSLab
lahari@DESKTOP-AIC33F7:~/OSLab$ mkdir shell_lab
lahari@DESKTOP-AIC33F7:~/OSLab$ cd shell_lab
lahari@DESKTOP-AIC33F7:~/OSLab/shell_lab$ pwd
/home/lahari/OSLab/shell_lab
lahari@DESKTOP-AIC33F7:~/OSLab/shell_lab$ nano shell_parser.c
lahari@DESKTOP-AIC33F7:~/OSLab/shell_lab$ gcc shell_parser.c -o shell_parser
lahari@DESKTOP-AIC33F7:~/OSLab/shell_lab$ ./shell_parser
Shell Parser
Enter command: hello world

Parsed output:
Argument 1: [hello]
Argument 2: [world]
lahari@DESKTOP-AIC33F7:~/OSLab/shell_lab$ ./shell_parser
Shell Parser
Enter command: hello\ world

Parsed output:
Argument 1: [hello world]
lahari@DESKTOP-AIC33F7:~/OSLab/shell_lab$ ./shell_parser
Shell Parser
Enter command: hello\;world

Parsed output:
Argument 1: [hello;world]
lahari@DESKTOP-AIC33F7:~/OSLab/shell_lab$ ./shell_parser
Shell Parser
Enter command: hello\&world

Parsed output:
Argument 1: [hello&world]
lahari@DESKTOP-AIC33F7:~/OSLab/shell_lab$ ./shell_parser
Shell Parser
Enter command: hello\|world

Parsed output:
Argument 1: [hello|world]
lahari@DESKTOP-AIC33F7:~/OSLab/shell_lab$ |

```

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_INPUT 1024
#define MAX_ARGS 100

```

```

int parse_command(char *input, char *args[])
{
    int argc = 0;
    int i = 0;
    int j = 0;

    char token[MAX_INPUT];

    while (input[i] != '\0')
    {
        // Skip spaces
        while (input[i] == ' ' || input[i] == '\t')
        {
            i++;
        }

        if (input[i] == '\0')
        {
            break;
        }

        j = 0;

        while (input[i] != '\0' &&
               input[i] != ' ' &&
               input[i] != '\t')
        {
            // Handle escaped character
            if (input[i] == '\\')
            {
                i++;

                if (input[i] != '\0')
                {
                    token[j++] = input[i++];
                }
            }
            else
            {
                token[j++] = input[i++];
            }
        }
    }
}

```

```

    token[j] = '\0';

    args[argc] = malloc(strlen(token) + 1);

    if (args[argc] == NULL)
    {
        perror("malloc");
        exit(1);
    }

    strcpy(args[argc], token);

    argc++;

    if (argc >= MAX_ARGS - 1)
    {
        break;
    }
}

args[argc] = NULL;

return argc;
}

int main()
{
    char input[MAX_INPUT];
    char *args[MAX_ARGS];

    while (1)
    {
        printf("myshell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
        {
            break;
        }

        input[strcspn(input, "\n")] = '\0';

        // Empty command
        if (strlen(input) == 0)

```

```

{
    continue;
}

// Exit shell
if (strcmp(input, "exit") == 0)
{
    break;
}

int argc = parse_command(input, args);

if (argc == 0)
{
    continue;
}

pid_t pid = fork();

if (pid < 0)
{
    perror("fork failed");
}
else if (pid == 0)
{
    // Child process

    execvp(args[0], args);

    // execvp only returns if an error occurs
    perror("Command execution failed");

    exit(1);
}
else
{
    // Parent process

    int status;

    waitpid(pid, &status, 0);

    if (WIFEXITED(status))
    {

```

```

        printf("Child exited with status %d\n",
               WEXITSTATUS(status));
    }
}

// Free arguments
for (int i = 0; i < argc; i++)
{
    free(args[i]);
}
}

printf("Shell terminated.\n");

return 0;
}

```

```

lahari@DESKTOP-AIC33F7:~/OSLab/shell_lab$ nano mini_shell.c
lahari@DESKTOP-AIC33F7:~/OSLab/shell_lab$ nano mini_shell.c
gcc shell_parser.c -o shell_parser
lahari@DESKTOP-AIC33F7:~/OSLab/shell_lab$ gcc mini_shell.c -o mini_shell
lahari@DESKTOP-AIC33F7:~/OSLab/shell_lab$ ./mini_shell
myshell> ls
mini_shell  mini_shell.c  shell_parser  shell_parser.c
Child exited with status 0
myshell> ls -l
total 48
-rwxr-xr-x 1 lahari lahari 16736 Sep 26 16:50 mini_shell
-rw-r--r-- 1 lahari lahari  2740 Sep 26 16:47 mini_shell.c
-rwxr-xr-x 1 lahari lahari 16528 Sep 26 16:49 shell_parser
-rw-r--r-- 1 lahari lahari  1650 Sep 26 16:43 shell_parser.c
Child exited with status 0
myshell> execvp(args[0], args);
Command execution failed: No such file or directory
Child exited with status 1
myshell> pwd
/home/lahari/OSLab/shell_lab
Child exited with status 0
myshell> echo Hello
Hello
Child exited with status 0
myshell> echo Hello World
Hello World
Child exited with status 0
myshell> echo Hello\ World
Hello World
Child exited with status 0
myshell> |

```